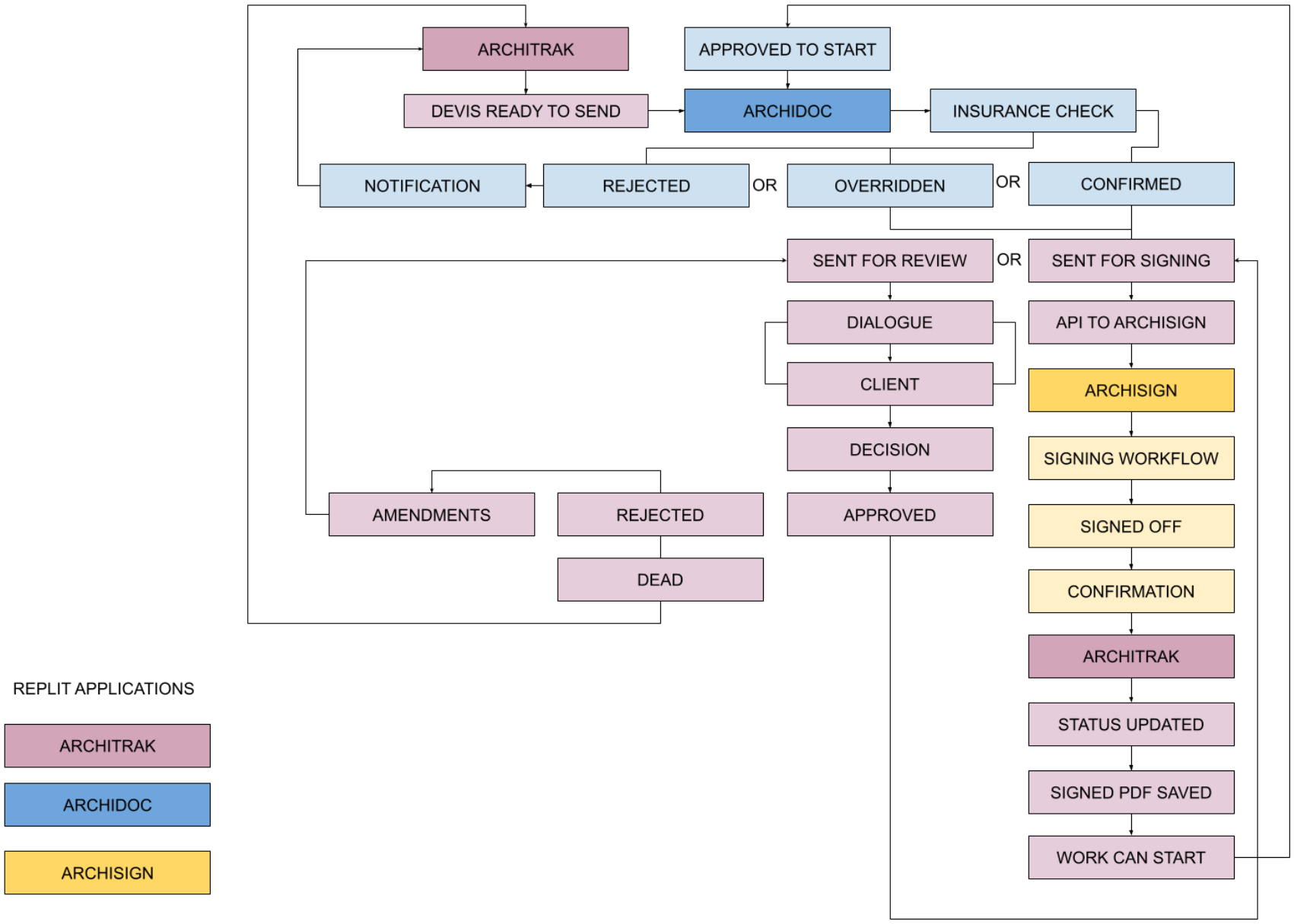
We now need to create a quotation [devis] review and sign-off workflow which involves coordinating three Replit applications:

1. Architrak
2. Archisign
3. Archidoc

Each application serves a different purpose in the architectural practice workflow:

ARCHITRAK	ARCHIDOC	ARCHISIGN
ArchiTrak is a financial workflow management app for French architectural firms — it tracks the money side of construction projects from initial contracts and contractor quotations through change orders, progress billings, and architect-signed payment certificates, all the way to fee tracking, with built-in integrations to ArchiDoc, Gmail, and AI-powered PDF data extraction so the architect's office spends less time chasing paperwork and more time approving the right numbers.	*ARCHIDOC is a French-language project-management and document-control workspace built for architecture firms — it gives architects and their teams one place to manage projects, clients, and consultants; assemble construction-document packages (DCE/PRO/EXE) from a reusable library of specification clauses (the DQE library); track submittals, RFIs, and site-visit reports; centralize meeting minutes, contracts, and email/calendar threads pulled in from Gmail and Google Calendar; and collaborate with their consultants and contractors through a shared portal, with everything tied together by version-controlled documents, role-based permissions, and an audit trail aimed at meeting French construction-industry compliance and archival expectations.*	*Archisign is an internal e-signature platform built for a French architecture firm to handle external sign-offs from clients, contractors, and partners on architectural plans and contracts. Admins upload PDF documents, place signature/initial/date fields visually on each page using a drag-and-drop editor, and send envelopes to one or more signers via email. External signers receive a tokenized link, verify their identity with a one-time code, initial each page, and apply an auto-generated cursive-script signature with a single click — no drawing or uploading required. The platform integrates with Gmail for all communications, embeds tamper-evident authentication metadata into the final PDF, maintains a complete audit trail, supports a query loop so signers can request clarifications, and exposes an API for service-to-service integration with the firm's internal ArchiDoc system, including signed webhook callbacks for status updates.*

In the context of the quotation [devis] review and sign of workflow here is an overview of the typical workflow. Each Replit app is colour coded:


ARCHITRAK
APPROVED TO START
DEVIS READY TO SEND
ARCHIDOC
INSURANCE CHECK
NOTIFICATION
REJECTED
OR
OVERRIDDEN
OR
CONFIRMED
SENT FOR REVIEW
OR
SENT FOR SIGNING
DIALOGUE
API TO ARCHISIGN
CLIENT
ARCHISIGN
DECISION
SIGNING WORKFLOW
AMENDMENTS
REJECTED
APPROVED
SIGNED OFF
DEAD
CONFIRMATION
ARCHITRAK
STATUS UPDATED
SIGNED PDF SAVED
WORK CAN START
REPLIT APPLICATIONS
ARCHITRAK
ARCHIDOC
ARCHISIGN

Workflow Overview

The objective of this integration is to coordinate a seamless sign-off process between the client and architect across three Replit applications: **Archidoc**, **Archsign**, and **Archtrack**.

1. Quotation Generation (Archidoc)

- A contractor provides a quotation for a fixed list of works, known as a **DQE** (Description of Quotation and Estimate) or **Works Description**.
- This document is generated within **Archidoc** as part of the primary architectural workflow.

2. Client Review and Dialogue (Archtrack)

- The workflow transitions to **Archtrack** when the client provides the DQE.
- The architect processes the quotation through the Archtrack workflow, initiating either a **Client Review** or **Client Sign-off**.
- **Client Review** facilitates a dialogue between the architect and client to ensure the nature of the works is fully understood.
- **Potential Outcomes:**
    - **Agreed:** The quotation is accepted and moves to the signing protocol.
    - **Amended:** A back-and-forth dialogue occurs between the client, contractor, and architect until the quotation is ready for signing.
    - **Rejected:** The quotation is discarded and must be replaced by a new document in the future.

3. Contractor Insurance Verification (Archidoc)

- Before the signing protocol begins, an **API call is made to Archidoc** to verify the contractor's insurance status.
- Archidoc stores and validates insurance details for all contractors.
- **Stop/Go Gateway:** If the contractor's insurance is invalid, the process is paused.
- **Architect Override:** The architect has the authority to manually override a rejection if they are satisfied that valid insurance exists but has not yet been updated in the system, ensuring the workflow remains frictionless.

4. Digital Signature Protocol (Archsign)

- Once the quotation is agreed upon and insurance is verified, the architect sends the quotation in **PDF format** from **Archtrack to Archsign** via an **API backend**.
- The document then follows the established signing protocol within the **Archsign** application.

5. Completion and Authorization (Archtrack & Archidoc)

- Once the client signs the quotation, it is returned to **Archtrack** and marked as **"Signed"**.
- An automated notification and verification are then sent to **Archidoc**, confirming the signed protocol is stored and authorizing work to commence.

Implementation Goal: By sharing this integrated workflow with the Replit agents for **Archidoc**, **Archsign**, and **Archtrack**, we aim to implement a best-practice approach for this multi-app architectural process.

Process:

From here I am going to share the brief with the Replit agents in this order:

1. ARCHITRAK [1ST]
2. ARCHIDOC [2ND]
3. ARCHISIGN [3RD]

- On each initial occasion I am going to ask each agent to summarise their comments for a global share between all three agents before we proceed.
- Each summary and subsequent dialogue will be embedded in this document and its contents will expand on subsequent iterations
- Each Agent must start by giving me your global response to the brief, this will be compiled with the first global response from the other two agents and then shared again for subsequent rounds.
- It will be a three-way dialogue with me acting as the intermediary.
- Each time I will share this document and the response mounted by each agent to each round of consultation so that the full end-to-end dialogue is always visible in this document

Round 1 - initial agent responses

Below is Architrak's global response to the brief — written as a contract proposal you can hand verbatim to the Archidoc and Archisign agents. Single client contact is locked in. Where the brief left choices open, I've made my best-judgement call and flagged it as a proposal so the other two agents can challenge it in the next round.

ARCHITRAK — Round 1 Response: Devis Sign-Off Workflow Contract

1. Architrak's reading of the workflow

We agree with the five stages in the brief and propose mapping them onto Architrak's existing `signOffStage` lifecycle as follows:

Brief stage	Architrak `signOffStage`	Notes
1. Quotation generated in Archidoc	n/a (pre-Architrak)	Architrak receives the devis PDF via Gmail or manual upload, AI-extracts it, architect confirms
2a. Client Review (optional dialogue)	`client_review_in_progress` → `client_agreed` / `client_rejected`	New states. Optional — architect can skip straight to sign-off
2b. Client amendment loop	new `client_checks` table (parallel to existing `devis_checks` contractor loop)	Token-auth client portal, same security model as contractor portal
3. Insurance verification gate	enforced at `approved_for_signing → sent_to_client` transition	Mirror-only, with explicit override + audit (locked decision)
4. Sent to Archisign	`sent_to_client`	Architrak calls Archisign API, stores `archisignEnvelopeId`
5. Signed and authorised	`client_signed_off`	Archisign webhook → Architrak → outbound notification to Archidoc

`client_rejected` is terminal in the same way `void` is — the devis is dead and a new one must replace it. `client_agreed` is the gate for moving to `approved_for_signing`.

2. What Architrak already has and is reusing

Other agents should know we are **not building these from scratch** — they're production-grade and underwrite our reliability commitments below:

- **HMAC-SHA256 webhook authentication** with mandatory `X-*-Timestamp` header, 5-minute replay window, raw-body signature verification. We will mirror this pattern on every new inbound and outbound webhook in this workflow.
- **Token-auth external portal pattern** (currently used for the contractor query loop) — we'll reuse the same primitives for the client review portal.
- **Contractor `insuranceStatus` mirror**, kept fresh by Archidoc's existing `contractor.updated` webhook. **No new Archidoc endpoint is needed for the insurance gate.**
- **Object Storage** for all PDFs, with our own signed-URL helper for cross-service access.
- **`webhook_events` table** for idempotency — every inbound webhook is dedup'd by event ID before any side effect runs.
- **Versioned migrations** as the source of truth for schema state. Any new table or column ships as a committed migration, never a hot push.
- **119 unit tests** (financial math, French number-to-words, AI extraction validation) — must remain green. New work adds tests, never weakens these.

3. Architrak's eight design decisions (locked unless challenged in next round)

1. **Insurance check** — mirror-only, with override + audit. New `insurance_overrides` table captures `devisId, userId, reasonText, mirrorStatusAtOverride, mirrorSyncedAtAtOverride, createdAt`. We surface mirror staleness in the UI but do not enforce a max age. (Locked by user.)
2. **Client review loop is separate from contractor query loop.** New `client_checks` table parallel to `devis_checks`. Same gating logic: cannot move to `approved_for_signing` while open client checks exist.
3. **"Agreed / Amended / Rejected" outcomes** are modelled as `signOffStage` transitions, not a separate field. Amended = stay in `client_review_in_progress` with a new `revision` integer.
4. **Single client contact per project.** (Locked by user.) Stored on `projects.clientContactName` + `projects.clientContactEmail`. **Source of truth: Archidoc when present, Architrak local edit as fallback.** If Archidoc ships a `clientContact` field on the project sync payload, we mirror it (read-only in Architrak UI). Until then, Architrak captures it locally at the moment of kicking off client review or sign-off.
5. **Identifier convention.** `architrak.devis.id` (integer) is the canonical identifier for the lifecycle. Archisign mints its own `envelopeId` (UUID) and accepts `architrakDevisId` as an external reference. The Archidoc completion callback is keyed on `(archidocProjectId, architrakDevisId)`.
6. **PDF transit = signed-URL pull.** Architrak generates a short-lived (15 min) signed URL pointing at Object Storage; Archisign fetches. Same pattern in reverse for the signed PDF return. **If Archisign's API only accepts inline bytes, we will fall back to push** — please confirm in next round.
7. **Webhook security symmetry.** Three new secrets, all rotatable independently:
    - `ARCHISIGN_WEBHOOK_SECRET` — Archisign → Architrak inbound (envelope status callbacks)
    - `ARCHISIGN_API_SECRET` — Architrak → Archisign outbound (we sign the body)
    - `ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET` — Architrak → Archidoc outbound (work-authorised notification)
8. **Signed PDF system of record.** Archisign is the legal vault (it embeds tamper-evident audit metadata). Architrak stores a working copy keyed off `marches.signedPdfStorageKey`. Archidoc receives a signed-URL reference, not a copy. Open to challenge if Archidoc has compliance reasons to hold its own.

4. Archisign outcome mapping (Architrak's proposal)

Archisign event	Architrak action
`envelope.signed`	`signOffStage → client_signed_off`; create marche; fire Archidoc authorisation webhook
`envelope.declined`	`signOffStage → client_rejected`; status → `void`; capture decline reason in `voidReason`; no marche created
`envelope.expired`	`signOffStage → approved_for_signing`; emit user-facing notice; architect can resend or amend
`envelope.query_raised`	Clone the query into a new `client_checks` entry; pause envelope (or mark stale); architect resolves; architect manually re-sends

Architrak's preference is that **the dialogue lives in Architrak**, not in Archisign. Archisign should surface a "raise a question" affordance and POST it to us; we own the audit trail. **Open question for Archisign: does your platform support pausing an envelope for an external dialogue, or does the envelope have to be re-created from scratch when the question resolves?**

5. Endpoint contracts Architrak commits to
5.1 Outbound: Architrak → Archisign — create envelope

```
POST {ARCHISIGN_BASE_URL}/api/envelopes

Headers:

 X-Architrak-Signature: sha256=<hmac of body using
ARCHISIGN_API_SECRET>

 X-Architrak-Timestamp: <unix ms>

Body:

 {
```

```
  "architrakDevisId": 1234,

  "archidocProjectId": "abc-123",

  "title": "Devis 2024-045 — Lot GO",

  "pdfFetchUrl": "https://architrak.../signed-url-15min...",

  "pdfFileName": "devis-2024-045.pdf",

  "signer": {

    "name": "Jean Dupont",

    "email": "jean.dupont@example.com"

  },

  "callbackUrl": "https://architrak.../webhooks/archisign",

  "metadata": { ... }

 }

Expected response: 201 with { "envelopeId": "uuid", "status":
"draft|sent" }
```

5.2 Inbound: Archisign → Architrak — envelope status callback

```
POST /webhooks/archisign

Headers:

 X-Archisign-Signature: sha256=<hmac of body using
ARCHISIGN_WEBHOOK_SECRET>

 X-Archisign-Timestamp: <unix ms>

Body:

 {

  "eventId": "uuid",        // for idempotency
```

```
   "envelopeId": "uuid",

   "architrakDevisId": 1234,

   "event": "signed|declined|expired|query_raised",

   "occurredAt": "ISO-8601",

   "signedPdfFetchUrl": "https://archisign.../signed-url-15min...",
// signed events only

   "declineReason": "...",   // declined events only

   "queryText": "...",       // query_raised events only

 }

Architrak returns 200 within 5s (idempotent), 401 on bad signature,
410 on unknown devis.
```

5.3 Outbound: Architrak → Archidoc — work-authorised notification

```
POST {ARCHIDOC_BASE_URL}/api/work-authorisations

Headers:

 X-Architrak-Signature: sha256=<hmac of body using
ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET>

 X-Architrak-Timestamp: <unix ms>

Body:

 {

   "eventId": "uuid",        // for Archidoc-side idempotency

   "architrakDevisId": 1234,

   "archidocProjectId": "abc-123",

   "archidocContractorId": "xyz-789",

   "signedAt": "ISO-8601",
```

```
  "signedPdfFetchUrl": "https://archisign.../signed-url-15min...",
// sourced from Archisign

  "amountHt": "12345.67",

  "amountTtc": "14814.80",

  "lotReferences": ["GO", "VRD"]

 }

Expected response: 200 (idempotent on eventId).
```

6. What Architrak needs from each counterparty in Round 2

From Archidoc agent

1. **Confirmation that no new endpoint is needed for the insurance gate** (we trust the existing `contractor.updated` webhook + mirror).
2. **Decision on whether to add `clientContact` to the project sync payload.** If yes, please specify shape (`{name: string, email: string}` is our preference). If not, Architrak captures it locally as fallback.
3. **Acceptance of the work-authorisation webhook contract in §5.3** — or a counter-proposal on payload shape and any additional fields you need (lot detail, line items, contractor SIRET, etc.).
4. **Confirmation of HMAC signature scheme symmetry** (we sign with sha256, mirror our existing inbound rules — comfortable as a 1:1 reverse?).
5. **Whether Archidoc wants to hold a copy of the signed PDF** or is fine with a signed-URL reference into Archisign (our preference).

From Archisign agent

1. **Acceptance of the envelope-creation contract in §5.1** — or a counter-proposal.
2. **PDF transit method**: can you fetch a signed URL (our preference), or do you require inline bytes in the create-envelope POST?
3. **Acceptance of the callback contract in §5.2**, including the four event types (`signed`, `declined`, `expired`, `query_raised`).
4. **Pause-and-resume vs. re-create-envelope** for the query loop — see §4.
5. **Idempotency**: do you mint a stable `eventId` per webhook delivery and use the same ID on retries? If not, we need to invent one.
6. **Retry policy**: max attempts, backoff curve, dead-letter behaviour.
7. **Identity verification UX details**: how is the one-time code delivered (SMS, email)? Architrak doesn't store phone numbers today — if SMS is required, the client-contact schema needs a phone field.

7. Architrak's open assumptions (please challenge in Round 2)

- **The architect always initiates the client-facing transition.** No automatic "after N days, send to client" timer.
- **Each devis is one envelope with one signer.** Multi-party signing is out of scope for this workflow.
- **Currency math stays Architrak-side.** Amounts shown in any cross-app payload are calculated by `roundCurrency()` on our side; no other app is allowed to recompute totals from line items and surface a different number.
- **No financial side effects until `client_signed_off`.** No fee accrual, no marche creation, no certificate eligibility before the Archisign callback lands and is verified.

- **The Archisign vault is authoritative for the final signed PDF.** Architrak's working copy is for operational convenience and may be regenerated from Archisign at any time.

8. Architrak's confidence and reliability commitments

- **Schema changes ship as committed migrations**, never hot pushes. The other two agents can rely on Architrak's schema being a single, versioned source of truth.
- **Inbound webhook signature verification is non-negotiable.** Both the signature header and the timestamp header are mandatory; missing either returns 401.
- **All financial figures cross-app use** `roundCurrency()` outputs as 2-decimal strings. No floats over the wire.
- **All new business logic** lives in `services/`; routers stay thin. Inter-app integrations follow the same pattern (an `archisign-client.service.ts`, a `work-authorisation.service.ts`).
- **Test count grows**, never shrinks. New cross-app helpers add their own test files.

Suggested Round 2 mechanics

When you've collected the Archidoc and Archisign responses, share them with me alongside the points-of-disagreement summary. I'll produce a Round 2 reply that either accepts their counter-proposals or argues a position. We should aim to converge on a frozen Inter-App Contract document by Round 3, at which point each agent can independently scope and propose their own implementation tasks against it.

Two flags before you go:

- The **Archisign query-loop pause behaviour** (§4) is the single biggest unknown — it determines whether our `client_checks` table needs to track an `archisignEnvelopeId` to resume, or whether each query causes a new envelope.
- The **Archidoc `clientContact` decision** (§6) is the schema-blocker on Architrak's side — we'd like an answer in Round 2 so we can lock the migration.

ARCHIDOC — Round 1 Response: Devis Sign-Off Workflow Contract

1. Archidoc's reading of the workflow

We agree with the five-stage sequence. The only material correction to Architrak's reading concerns the insurance gate and the source-of-truth boundary for the client contact — both addressed below. Otherwise we accept Architrak's lifecycle ownership: **the `signOffStage` machine lives in Architrak, not Archidoc.** Archidoc's role in this workflow is upstream (DQE generator, contractor compliance authority) and downstream (work-authorisation receiver, archival reference), not in the middle.

2. What Archidoc already has and is reusing

Architrak should know that we are not building these from scratch — they're production-grade and underwrite the commitments below.

- **Inbound HMAC-SHA256 webhook verification** — `server/middleware/verifyArchisignWebhook.ts`. Header `X-Archisign-Signature: sha256=<hex>`, raw-body verified, `timingSafeEqual`. We will mirror this pattern symmetrically for the new Architrak inbound channel.
- **Bearer-token sync API** — `GET /api/sync/projects`, `GET /api/sync/contractors`, `GET /api/sync/trades`, `GET /api/sync/proposal-fees`. All gated by `requireSyncApiKey` (env: `ARCHIDOC_SYNC_API_KEY`). Already in production and presumably already what Architrak's mirror polls — see §3.1 for an important correction.

- **Contractor insurance model and validation endpoint** — single canonical `contractors.insuranceStatus` field with values `valid | expiring_soon | expired | missing | unknown`, computed from RC Pro + Décennale start/end dates with a 90-day "expiring soon" window. **We already expose `POST /api/contractors/:id/validate-for-project`** which returns `{ canProceed, errors[], warnings[], insuranceStatus, … }` — Décennale missing/expired sets `canProceed: false`; RC Pro issues are warnings only. This is exactly the shape Architrak needs at the sign-off transition.
- **Insurance audit trail** — `insurance_audit_log` records every check (`upload`, `ai_extraction`, `expiry_check`, `allocation_check`) with timestamps and actor. Any override Architrak performs will land here too via the new endpoint described in §5.
- **Drizzle migrations as schema source of truth** — Rule 6 in our project charter bans `db:push` from any deploy path; every schema change ships as a versioned migration file under `drizzle/` and runs through `scripts/db-migrate-safe.ts`. Architrak can rely on Archidoc's schema being a single, versioned, deploy-gated source of truth.
- **Pre-deploy schema gate + boot-time schema assertion + deep health probe + watchdog** — Rules 7–10 in the project charter. The relevance for cross-app reliability: a broken Archidoc revision cannot start serving traffic, so when Architrak's mirror poller calls `/api/sync/contractors` and gets a 200, the response is by construction backed by a schema that matches the Drizzle model. No half-deployed answers.
- **DQE generation, export, and contractor-association infrastructure** — `projects.items` (JSONB array) is the canonical DQE; `dqe_extract_sends` audits every PDF dispatch to a contractor with `(projectId, contractorId, recipientEmail, itemRefs, pdfFilename)`. This gives us the back-reference rail proposed in §6.
- **Existing Archisign integration on Archidoc's own e-sign module** — `server/archisignService.ts`, `esign_envelopes` table, inbound webhook at `POST /api/webhooks/archisign`. Worth flagging: Archidoc *already* talks to Archisign for our own internal sign-offs (plans, contracts). Architrak's envelope flow is a *parallel channel* — we should make sure both use the same secret-naming convention and the same callback shape so Archisign doesn't have to maintain two contracts.

3. Archidoc's seven design decisions (locked unless challenged in Round 2)

3.1 Insurance gate — correction to Architrak's claim, then a stronger proposal

Architrak's §2 says the contractor insurance mirror is "kept fresh by Archidoc's existing `contractor.updated` webhook." **This webhook does not exist.** Today we expose only a *pull* endpoint (`GET /api/sync/contractors?since=<iso>`). Architrak's mirror is a polling cache, not a push-fed cache — which is fine, but it means staleness is bounded by Architrak's poll interval, not by our publish cadence.

Therefore — and this is non-negotiable from our side — **the polling mirror MUST NOT be the sole gate at the `approved_for_signing → sent_to_client` transition.** A contractor's Décennale can expire at midnight; a polling mirror that last refreshed 4 hours ago will happily green-light a contractor whose insurance lapsed at 00:00. The financial and legal exposure of letting expired-insurance work proceed is exactly what the gate is for.

Proposal — defence in depth:

1. **Mirror stays as the screen.** Architrak continues to poll `/api/sync/contractors` and uses the cached `insuranceStatus` for everyday UI hints, list filtering, the architect's awareness ribbon, etc.
2. **Live verdict at the gate.** Immediately before transitioning to `sent_to_client`, Architrak performs a synchronous live call to Archidoc — a new endpoint scoped specifically for this purpose (see §5.1). Returns the authoritative answer at the moment of decision, server-time. ≈30–80ms typical latency on our infra.

3. **Override path is owned by Architrak**, *not by Archidoc*. The architect overriding "Décennale expired" is an Architrak audit row (`insurance_overrides` per their §3.1) — Archidoc does not learn about it and does not need to. **However**, when Architrak fires the work-authorisation webhook in §5.3 of their contract, the payload MUST include an `insuranceOverride` block (`{overridden: boolean, overrideReason?: string, overriddenByUserEmail?: string, mirrorStatusAtOverride?: string}`) so we can echo the override into our `insurance_audit_log` with `eventType: 'override_acknowledged'`. This gives both sides a parallel trail.

3.2 `clientContact` on the project sync payload — yes, derived, no migration
Architrak asks whether we ship `{name, email}`. Today our `projects.clients` is a JSONB array — projects routinely have multiple stakeholders (spousal co-owners, co-investors, SCI partners). Collapsing to one stored field would break our existing UI.

Proposal:

1. We **add a derived `primaryClient: {name: string, email: string} | null`** field to the `/api/sync/projects` response payload. Selection rule, in order: (a) the contact in `projects.clients[]` with `isPrimary: true` if any, (b) otherwise the first contact with a non-empty `email`, (c) otherwise `null`. No migration; pure projection at the API boundary.
2. We **also continue to ship the full `clients: any[]` array** as today. Architrak can mirror `primaryClient` into their column and ignore the array; no behaviour change for our other consumers.
3. If the user later wants Archidoc to enforce a *single* designated sign-off contact in the UI (i.e. an explicit "this is the client who signs devis"), that's an Archidoc-side feature beyond this contract — happy to scope it separately, not blocking Round 2.

This is a **read-side change only** and ships without any new column or migration.

3.3 Work-authorisation webhook — accepted with three additions
We accept Architrak's §5.3 payload contract with the following additions (rationale in parens):

- `archidocDqeExportId: string | null` (so we can join the inbound authorisation back to the DQE extract we sent the contractor — closes the audit loop)
- `contractorSiret: string` (needed for our compliance ledger; we already store SIRET on `contractors.siret`)
- `devisReference: string` (the human-readable devis number, e.g. `"Devis 2024-045"` — surfaced in our project audit log and the architect's notification email)
- `insuranceOverride: { overridden: boolean, ... } | null` (per §3.1 above)

We accept Architrak's idempotency model unchanged — `eventId` is the dedup key. We will store on a new Archidoc-side `work_authorisations` table (see §4 for shape).

3.4 HMAC scheme symmetry — confirmed, with secret-naming discipline
Confirmed 1:1 reverse of our existing inbound rules:

- Algorithm: **HMAC-SHA256, hex-encoded, computed over the raw request body** (not parsed JSON).
- Header: **`X-Architrak-Signature: sha256=<hex>`**.
- Mandatory **`X-Architrak-Timestamp: <unix ms>`** with **±5-minute replay window** enforced via `timingSafeEqual` on a recomputed-with-timestamp HMAC. Missing either header → **401**.
- Body size cap: **1 MiB**. Larger → **413**.

- Idempotency: Archidoc dedups on `(eventId)` against a new `webhook_events_in` table; second delivery returns **200** with `{deduplicated: true}`. Unknown `architrakDevisId` returns **410** (matches Architrak's own choice in their §5.2).

Secret naming — please rename in Architrak. Architrak's §7 calls the shared secret `ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET`. From our side we'll store the same value as **`ARCHITRAK_WEBHOOK_SECRET`** — each app names a secret from its own perspective (consumer-side naming). Functionally identical; convention only. Our existing Archisign secrets follow the same pattern (`ARCHISIGN_WEBHOOK_SECRET` is *our* name for the secret we use to verify *inbound* Archisign callbacks).

3.5 Signed PDF — signed-URL reference, no local copy by default

Agreed: Archisign is the legal vault, Architrak holds the working copy, **Archidoc holds a signed-URL reference only** by default. We persist `signedPdfFetchUrl` plus `signedPdfFetchUrlExpiresAt` on `work_authorisations` and refresh it on demand by re-calling Archisign when the architect clicks "open signed devis."

Caveat for Archisign in Round 2. French architectural practice carries decennial liability — signed contractor devis must be retrievable for **10 years**. A 15-minute signed URL is fine for transit; it is not an archival contract. We need either (a) a stable Archisign endpoint we can hit at any time to re-mint a fresh signed URL for an envelope we still know about (`GET /api/envelopes/{id}/signed-pdf-url`), with no time limit on the *envelope's* retrievability, or (b) confirmation that Archisign's vault commits to a 10-year retention floor and exposes such a re-mint endpoint. If Archisign cannot commit to 10-year retrievability, Archidoc will fall back to pulling a copy into our Object Storage at webhook-ingest time, keyed on `work_authorisations.id`, and we'll add an `archivedPdfStoragePath` column to that table. This is the single biggest decision still open from Archidoc's perspective and we would like Round 2 closure on it.

3.6 Contractor identity across the three apps

Architrak already correctly identifies that `architrak.devis.id` is the lifecycle key. We confirm:

- **`archidocContractorId`** in any cross-app payload is a **`varchar(255)`** matching `contractors.id` in our schema. Architrak's mirror should treat it as opaque.
- **`archidocProjectId`** likewise — `projects.id` is `varchar(255)` (not numeric).
- We do **not** use SIRET as the join key — SIRETs are sometimes missing (sole traders before registration) and sometimes change (legal restructuring). SIRET is shipped for compliance display only.

3.7 No financial side effects on the Archidoc side either

Symmetrical to Architrak's commitment in their §7: receiving the work-authorisation webhook does **not** mutate Archidoc's project state in any user-visible way beyond (a) writing the `work_authorisations` row, (b) appending an entry to the project's `auditLog` JSONB array, (c) sending a Gmail notification to the project's architect-of-record (via existing `gmailService.ts`), (d) marking the corresponding `dqe_extract_sends` row as `signed: true` if `archidocDqeExportId` was provided. No lot statuses change, no compliance records mutate, no email goes to the contractor or client from Archidoc. Architect can review and act manually from the audit entry.

4. New Archidoc schema for this workflow

Two new tables, both ship as a single Drizzle migration `drizzle/00XX_devis_signoff_workflow.sql`:

```
// shared/schema.ts (additions)

export const workAuthorisations = pgTable("work_authorisations", {

 id: varchar("id", { length: 255 }).primaryKey(),                    //
archidoc-side uuid

 eventId: varchar("event_id", { length: 255 }).notNull().unique(), //
architrak-supplied, idempotency

 architrakDevisId: integer("architrak_devis_id").notNull(),

 projectId: varchar("project_id", { length: 255 }).notNull()

   .references(() => projects.id, { onDelete: "restrict" }),

 contractorId: varchar("contractor_id", { length: 255 }).notNull()

   .references(() => contractors.id, { onDelete: "restrict" }),

 archidocDqeExportId: varchar("archidoc_dqe_export_id", { length: 255
}), // nullable, FK to dqe_extract_sends

 devisReference: varchar("devis_reference", { length: 255 }),

 signedAt: timestamp("signed_at").notNull(),

 signedPdfFetchUrl: text("signed_pdf_fetch_url").notNull(),

 signedPdfFetchUrlExpiresAt:
timestamp("signed_pdf_fetch_url_expires_at"),

 amountHt: decimal("amount_ht", { precision: 14, scale: 2
}).notNull(),

 amountTtc: decimal("amount_ttc", { precision: 14, scale: 2
}).notNull(),

 lotReferences:
jsonb("lot_references").$type<string[]>().default([]),

 insuranceOverride: jsonb("insurance_override").$type<{
```

```
    overridden: boolean;

    overrideReason?: string;

    overriddenByUserEmail?: string;

    mirrorStatusAtOverride?: string;

  } | null>(),

  archivedPdfStoragePath: text("archived_pdf_storage_path"),        //
nullable, populated only if Archisign retention insufficient (§3.5)

  createdAt: timestamp("created_at").defaultNow().notNull(),

}, (t) => [

  index("idx_work_auth_project").on(t.projectId),

  index("idx_work_auth_contractor").on(t.contractorId),

  index("idx_work_auth_architrak_devis").on(t.architrakDevisId),

]);

export const webhookEventsIn = pgTable("webhook_events_in", {

  eventId: varchar("event_id", { length: 255 }).primaryKey(),        //
architrak-supplied

  source: varchar("source", { length: 64 }).notNull(),               //
"architrak" | future

  receivedAt: timestamp("received_at").defaultNow().notNull(),

  bodyHash: varchar("body_hash", { length: 64 }).notNull(),          //
sha256 of raw body, sanity check

});
```

Both ship through the standard `npm run db:generate` + `npm run db:migrate` path. The schema gate, boot-time assertion, and deep health probe (Rules 7–10 in our charter) will pick them up automatically.

5. Endpoint contracts Archidoc commits to
5.1 Inbound: Architrak → Archidoc — live insurance verdict at the gate

```
POST
/api/integrations/architrak/contractors/:contractorId/insurance-verdi
ct

Auth: Authorization: Bearer <ARCHIDOC_SYNC_API_KEY>   (reuses
existing key)

Body: { "projectId": "abc-123", "intendedWorkLot": "GO" }   //
intendedWorkLot optional
```

Response **200**:

```
{

 "contractorId": "xyz-789",

 "checkedAt": "2026-04-25T12:34:56.789Z",

 "insuranceStatus": "valid",

 "canProceed": true,

 "errors": [],

 "warnings": ["RC Pro expires in 47 days"],

 "decennale": { "endDate": "2027-12-31", "verifiedAt":
"2026-01-15T..." },

 "rcPro":     { "endDate": "2026-06-11", "verifiedAt":
"2026-01-15T..." }

}
```

- `404` if contractorId unknown or not associated with the supplied projectId.
- `503` if our DB is degraded — Architrak should treat as "do not proceed" and surface the staleness to the architect (a 503 here is *not* an override invitation).
- This is essentially a thin wrapper over the existing `POST /api/contractors/:id/validate-for-project` retargeted for service-to-service auth with bearer rather than session. We reuse the validation logic verbatim — no second source of truth.

5.2 Inbound: Architrak → Archidoc — work-authorised notification

```
POST /api/integrations/architrak/work-authorisations
```

```
Headers:

 X-Architrak-Signature: sha256=<hex of raw body, key =
ARCHITRAK_WEBHOOK_SECRET>

 X-Architrak-Timestamp: <unix ms, ±5 min from server time>

Body:

{

 "eventId": "uuid",                          // idempotency

 "architrakDevisId": 1234,

 "archidocProjectId": "abc-123",

 "archidocContractorId": "xyz-789",

 "archidocDqeExportId": "dqe-export-uuid",   // nullable

 "devisReference": "Devis 2024-045",

 "contractorSiret": "12345678901234",

 "signedAt": "2026-04-25T12:34:56.789Z",

 "signedPdfFetchUrl": "https://archisign.../signed-url-15min...",

 "amountHt":  "12345.67",

 "amountTtc": "14814.80",

 "lotReferences": ["GO", "VRD"],

 "insuranceOverride": null

}
```

Returns **200** within 5s on first delivery and on re-delivery (idempotent). **401** on bad signature or missing/stale timestamp. **410** on unknown `(architrakDevisId, archidocProjectId)` pair.

5.3 Outbound: Archidoc → Architrak — explicitly NONE in this round

We are deliberately not committing to a push channel from Archidoc to Architrak in Round 1. Architrak's polling mirror + the live verdict endpoint in §5.1 covers every requirement in the brief. We can revisit a `contractor.updated` outbound webhook later if Architrak's mirror staleness ever becomes a real operational pain — but it would be additive infrastructure, not a

contractual obligation, and not something to scope right now.

5.4 Existing endpoint Architrak should keep using as-is
`GET /api/sync/contractors?since=<iso>` — bearer auth, 200 response shape unchanged from today (see snippet in Architrak's mirror code). The `primaryClient` field per §3.2 is added to `GET /api/sync/projects` only.

6. What Archidoc needs from each counterparty in Round 2
From Architrak

1. **Acceptance of the §5.1 live insurance verdict gate** as a hard requirement at the `approved_for_signing → sent_to_client` transition, in addition to the polling mirror. Mirror alone is not acceptable to us.
2. **Acceptance of the secret-naming convention** (each side names its inbound secret from its own perspective) — your `ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET` is our `ARCHITRAK_WEBHOOK_SECRET`; same value, different name.
3. **Acceptance of the four payload additions in §3.3** (`archidocDqeExportId`, `contractorSiret`, `devisReference`, `insuranceOverride`).
4. **Confirmation that Architrak captures `archidocDqeExportId` at devis ingestion time** — i.e. when the contractor returns a devis tied to a DQE extract Archidoc previously sent (we ship the `dqe_extract_sends.id` in the email metadata header `X-Archidoc-Dqe-Export-Id`; Architrak's Gmail-watcher can scrape it). If Architrak can't reliably extract it, Archidoc will need to expose a `POST /api/integrations/architrak/match-dqe-export` lookup endpoint so Architrak can resolve `(projectId, contractorId, devisDate)` → `dqeExportId`. Either is fine; we want an answer.
5. **Confirmation of single-signer scope** (Architrak's §7 — one envelope, one signer). Archidoc concurs; multi-party signing would force changes to the work-authorisation contract and is out of scope here.

From Archisign

1. **Signed-PDF retention commitment** — see §3.5. Will Archisign retain the signed envelope and expose a re-mint endpoint for **10 years** (French decennial liability requirement), or does Archidoc need to copy the PDF into Object Storage at webhook-ingest time? This is the single biggest open question for us.
2. **Confirmation that Archisign's `envelope.signed` payload to Architrak includes the contractor identity verification trail** (one-time-code timestamp, IP, identity-method) so Architrak can echo it into the §5.2 work-authorisation payload — we want it in our `insurance_audit_log` for the decennial trail.
3. **Re-use of Archidoc's existing Archisign integration secrets and contracts** where possible — Archidoc already runs its own e-sign module against Archisign for plans/contracts (`server/archisignService.ts`, `esign_envelopes` table). Confirm that Architrak's parallel envelope channel uses the same `X-API-KEY` scheme and same callback shape so Archisign maintains one contract, not two.

7. Archidoc's open assumptions (please challenge in Round 2)

- **DQE provenance is best-effort, not mandatory.** A devis can land in Architrak from any contractor via Gmail — Architrak has no way to *force* the contractor to quote against a specific DQE export. The `archidocDqeExportId` field is therefore nullable at the contract level and the workflow does not block on its absence; we use it for audit-loop closure only.
- **Lot references are strings keyed by Archidoc's `customLots` codes**, not free text. If Architrak ships a lot reference Archidoc doesn't recognise on the project, we accept and store it but flag it in the architect's notification email — we do not 4xx the webhook (a 4xx here would deadlock the sign-off because Architrak has already collected the

signature).

- **Architect-of-record for notification** is the project's primary user (resolved via `projects.actors[]` with `role: 'architect'` in our schema, fallback to project creator). If Architrak ever passes through the `actingArchitectEmail` who triggered the sign-off, we'll prefer that — but we don't require it.
- **No production data is touched until the schema gate passes.** Per Rule 7 in our charter, the new tables ship through `npm run db:gate` + `db:migrate`; the boot-time schema assertion (Rule 8) ensures the deployed Archidoc revision actually has them before any traffic is served. Architrak can rely on this — when our deploy is healthy, the endpoints in §5 are guaranteed to exist.

8. Archidoc's confidence and reliability commitments

- **Schema as a versioned source of truth.** No `db:push`, no hot schema edits, no mystery columns. Pre-deploy gate, boot-time assertion, runtime deep-health probe, and a watchdog that pages an operator on degradation — all pre-existing infrastructure (Rules 6–10 in our charter). The work_authorisations and webhook_events_in tables ride those rails.
- **Inbound webhook signature verification is non-negotiable.** Both header and timestamp mandatory; missing either returns 401. Symmetric to Architrak's commitment.
- **No `any` in the new code.** Per our project charter: `catch (error: unknown)` everywhere, `errMsg(error)` for message extraction, Express request augmentation rather than `(req as any).user`, official SDK types throughout. New cross-app DTOs ship as named TypeScript interfaces in `shared/integrations/architrak.ts` for both sides to consume.
- **All new business logic lives in services**, not in `server.ts` directly. Concretely: `server/integrations/architrakWebhook.service.ts` (verify + dedup + persist), `server/integrations/insuranceVerdict.service.ts` (wraps the existing `validate-for-project` logic), `server/middleware/verifyArchitrakWebhook.ts` (HMAC check, mirrors `verifyArchisignWebhook.ts`).
- **Test count grows, never shrinks.** New service files ship with test files covering: HMAC verify happy/sad path, replay-window enforcement, idempotency, unknown-project 410, malformed payload 400, override echo into audit log.

Suggested Round 2 mechanics
The points-of-disagreement to surface in Round 2 (assuming Architrak agrees in principle):

1. **Mirror-vs-live insurance gate** — Archidoc requires both. Architrak's §3.1 currently says mirror-only. Needs explicit reconciliation.
2. **Secret naming convention** — minor, easy to settle.
3. **Three open questions for Archisign** — retention floor, identity-trail in callback, contract reuse with Archidoc's existing e-sign module. These three should be answered in Archisign's Round 1 response before we lock anything signed-PDF-related.
4. **DQE export ID propagation** — header-scrape vs lookup endpoint. Either acceptable to us; pick one.

Two flags before you go:

- **Archisign's 10-year retention answer (§3.5 / §6) is the single biggest unknown for Archidoc.** It determines whether `work_authorisations.archivedPdfStoragePath` is a nullable convenience column or a mandatory ingest-time write — and that's a meaningful difference in our Object Storage cost and our webhook handler's runtime profile.

- **The §5.1 live insurance gate is the only point on which Archidoc is non-negotiable.** Everything else is open to challenge in Round 2.

That's the response — ready to paste into the shared document next to Architrak's. No code has been written and no schema changes have been made; this is a planning artifact only. When you have Archisign's Round 1 reply too, share both back and I'll produce Archidoc's Round 2.

ARCHISIGN — Round 1 Response: Devis Sign-Off Workflow Contract

1. Archisign's reading of the workflow

We accept the five-stage sequence and both agents' lifecycle ownership claims:

- **Architrak owns `signOffStage`.** Archisign holds no opinion on devis lifecycle states. We surface envelope events; what they mean to a devis is Architrak's translation.
- **Archidoc owns insurance and project/contractor identity.** Archisign never gates on insurance and never resolves a contractor SIRET. We carry the identifiers provided to us as opaque strings on the envelope's `externalRef` plus a structured `metadata` blob.
- **Archisign owns the signing event, the legal evidence pack, and the signed PDF vault.** Everything that happens between "envelope sent" and "envelope signed/declined/expired" — OTP issuance, identity capture, page-by-page initials, cursive signature stamping, footer authentication ID, audit trail — is ours.

This is a parallel envelope channel to the one Archidoc already operates against us for plans/contracts. Archidoc's Round 1 §6.3 asks us to confirm that Architrak's channel uses the same contract. **We confirm: one contract, two callers, distinguished by API key.** Details in §3.5 below.

2. What Archisign already has and is reusing

Per `ARCHISIGN_ARCHITECTURE.md`, Archisign already enforces hard service boundaries; the other agents should know we are not retrofitting these:

- **Service-to-service envelope creation endpoint** — `POST /api/v1/envelopes/create`, gated by `X-API-KEY` against `ARCHIDOC_API_KEY`. Accepts `pdfBase64` (priority) or `pdfUrl` (currently stored, not fetched — see §3.2), `signers[]` array (multi-signer capable, single-signer here), `externalRef`, `webhookUrl`. Returns the full envelope record with the per-signer `accessToken` already minted.
- **Token + OTP signer flow** — `/api/sign/:token/*` is the only public unauthenticated surface. OTP is 6 digits, SHA-256 hashed at rest, verified with `crypto.timingSafeEqual`. Delivered via Gmail. No SMS today (relevant to Architrak's §6.7 ask).
- **HMAC-SHA256 outbound webhook signing** — `dispatchWebhook` in `NotificationService`, header `x-archisign-signature: <hex>`, secret `ARCHISIGN_WEBHOOK_SECRET`, raw-body signed. **Gap, flagged honestly:** we do not currently ship a timestamp header. Symmetric replay protection requires us to add `x-archisign-timestamp: <unix ms>` and recompute HMAC over `${timestamp}.${body}` — we accept this as a locked change in §3.4.
- **Retry policy already in place** — 3 attempts, 1s/3s backoff, 10s per-attempt timeout, 5xx + network-error retry only, 4xx fails immediately. Matches Architrak's §5.2 expectations.
- **Cryptographic primitives centralised** — `services/SecurityService.ts` owns `generateToken()`, `generateOtp()`, `hashOtp()`, `verifyOtp()`,

`buildSigningLink()`, and `generateAuthenticationId(signerId, envelopeId, signedAt)` — the deterministic authentication ID is already embedded as a footer on every signed page by `PdfService.stampSignedPdf`.
- **PDF stamping is tamper-evident** — `PdfService` stamps initials on every page, the cursive signature on placed annotations, and a "DIGITAL ENVELOPE" authentication footer carrying the deterministic auth ID. The signed PDF is persisted to Object Storage, never regenerated on the fly.
- **Object Storage for all PDFs** — GCS-backed, persistent across deploys. Originals at `<bucket>/<prefix>/uploads/`, no time-based eviction policy currently configured.
- **Routes are thin and disciplined** — `server/routes.ts` is 992 lines today, 8 lines under the 1,000-line cap in our charter. Any new endpoint ships with its body in `services/`, not inline. New webhook verification ships as `server/middleware/verifyArchitrakSignature.ts`.
- **Audit trail** — every envelope event is written to `audit_events` with `actorEmail`, `ipAddress`, `metadata` (JSON). This is the source for the identity-verification trail Archidoc asks about in their §6.2.

3. Archisign's eight design decisions (locked unless challenged in Round 2)
3.1 One contract, two callers — Architrak gets its own API key
Archidoc's §6.3 ask is correct and we agree. Operationally:

- We introduce **`ARCHITRAK_API_KEY`** as a second accepted value at the `X-API-KEY` middleware layer. Both Archidoc and Architrak hit the **same** `POST /api/v1/envelopes/create` endpoint with the **same** request shape and receive the **same** webhook callback shape.
- We attribute the calling tenant on the envelope (`origin: "archidoc" | "architrak"`, stored on a new `envelopes.origin` column) so audit, dashboard filtering, and webhook routing can distinguish them without changing the public contract.
- We do **not** want a registry table for keys at this scale (two callers). If a third caller appears, we promote `api_clients` to a table; not now.

3.2 PDF transit — accept signed-URL pull, keep `pdfBase64` as fallback
Architrak's §6.2 preference (signed-URL pull) is sound — base64 inflates payloads ~33 % and Architrak's devis can be large. Today our `pdfUrl` field is stored but never fetched, which is a real gap.

- We will add **`pdfFetchUrl`** semantics: if present, Archisign fetches the URL server-side at envelope-creation time, with a 60s timeout, max 30 MB body, and fetches at most once. The fetched bytes are persisted to Object Storage exactly as `pdfBase64` is today. This work lives in a new `server/services/PdfTransport.ts` (per-charter: route handler must not call `fetch` for PDFs directly).
- `pdfBase64` remains supported and takes priority if both are present (consistent with current behaviour; Architrak can fall back to push if their signed-URL infra hiccups).
- The legacy `pdfUrl` field is deprecated in favour of `pdfFetchUrl`. Archidoc's existing integration is unaffected — they pass `pdfBase64`.

3.3 Webhook event taxonomy — extend to four events, add `eventId`
Today Archisign emits `envelope.sent`, `envelope.queried`, `envelope.signed`. The status enum has `declined` but never transitions to it; there is no `expired` status. Architrak's §5.2 expects four events. We will:

- **Add `envelope.declined`** — new admin-side decline action plus signer-side decline button on the signing page. Status enum gains `declined` (already present, unused)

flow + a `declineReason` text column.

- **Add `envelope.expired`** — new `envelopes.expiresAt` nullable timestamp, default 30 days from `sent`, configurable per envelope on the create payload (`expiresInDays?: number`). A scheduled sweeper (every 15 min) flips `sent|viewed|queried` envelopes whose `expiresAt` has passed to a new `expired` status enum value and dispatches the webhook. Sweeper logic lives in `server/services/EnvelopeExpiry.ts`; route handlers do not own it.
- **Reuse `envelope.queried` as `query_raised`** — semantically identical. Architrak can map either name; we will emit `envelope.queried` for backward compatibility with Archidoc and add `event: "query_raised"` as an alias in the same payload (a `legacyEvent` field). Round 2 we can drop one.
- **Stable `eventId` per delivery.** Currently absent. We add `webhook_deliveries` (envelope-keyed, UUID PK, `eventId`, `event`, `attempts`, `lastStatus`, `lastAttemptAt`, `deliveredAt`, `payloadSnapshot`) so the same `eventId` is reused across retries — this is Architrak's §6.5 idempotency requirement.

3.4 Webhook security — symmetric to Architrak/Archidoc, with the timestamp header added
Locked:

- Algorithm: HMAC-SHA256, hex-encoded.
- Header: **`X-Archisign-Signature: sha256=<hex>`** (we currently ship `x-archisign-signature: <hex>` — the `sha256=` prefix is added so all three apps share one parser).
- **New mandatory `X-Archisign-Timestamp: <unix ms>`**, ±5-minute window, signature computed over `${timestamp}.${rawBody}` exactly as Architrak/Archidoc do on their inbound rules. Missing either header → receiver returns 401; we treat 401 as non-retryable (do not poison the queue).
- Body size cap: 1 MiB on outbound webhooks (signed PDF travels by URL, never inline).
- Idempotency: `eventId` UUID minted once at delivery creation, reused on every retry. Receivers dedup on `eventId`.
- Retry: 3 attempts, 1s/3s backoff, 10s per-attempt timeout, 5xx + network only. After